CSA06 – Design and Analysis of Algorithms

CO1 Assessment Tool 1: IT Application-Based Problem Solving

Scalability Audit of an E-Commerce Platform — ShopSphere

Given:

- Current catalogue size: $n = 100{,}000$
- Expected catalogue size: $n = 1{,}000{,}000$
- Server processing speed: 10^8basic operations per second
- Nightly processing window: 10 minutes $= 600$seconds

The number of basic operations the server can perform in 10 minutes is:

$$10^8 \times 600 = 6 \times 10^{10}$$

Therefore, the platform can execute approximately **60 billion basic operations within the nightly processing window**, assuming the stated processing rate applies uniformly.

(a) Understand the Problem

For each module, we identify the input size, basic operation, location of the operation, and expected growth rate.

Module	**Input size**	**Basic operation**	**Where it occurs**	**Expected growth**
M1: Duplicate-Listing Detector	$n$products	Comparing two product titles and sellers	Inside the nested loops	Quadratic: $\Theta(n^2)$
M2: API Rate Limiter	$n$requests	Executing Audit(Request[j])	Inside the inner loop	Linear: $\Theta(n)$
M3–S1: Sequential Search	$n$products	Comparing the search key with a product	During sequential scanning	Best: $\Theta(1)$; average and worst: $\Theta(n)$
M3–S2: Binary Search	$n$sorted products	Comparing the search key with the middle entry	At each recursive or iterative step	$\Theta(\log n)$

Module	Input size	Basic operation	Where it occurs	Expected growth
M4: Current Recommendation Engine	$n$customer–product records	Processing and merging partial results	Recursive calls and linear merge	$\Theta(n^2)$
M4: Proposed Recommendation Engine	$n$customer–product records	Processing and merging partial results	Recursive calls and linear merge	$\Theta(n \log n)$
M5: Sales Analytics Engine	$n$sales records	Sorting and merging revenue records	At each recursion level	$\Theta(n\log^2 n)$

Explanation

M1: Each product is compared with every product appearing after it. The number of comparisons increases quadratically.

M2: The inner loop executes 1,2,4,8, ...operations. These values form a geometric series, so the total is linear rather than $n \log n$.

M3–S1: Sequential search examines products one by one. Its execution count depends on where the target appears or whether it exists.

M3–S2: Binary search halves the remaining search space after each comparison. Therefore, the number of comparisons grows logarithmically.

M4: The current design makes four recursive calls on subproblems of size $n/2$. Removing replication reduces the number of recursive calls to two.

M5: The recurrence combines two recursive calls with an $n \log n$additional cost, producing an overall complexity of $\Theta(n\log^2 n)$.

(b) Analyse the Iterative Modules

M1: Duplicate-Listing Detector

Step 1: Identify the basic operation

The basic operation is the comparison of the titles and sellers of two products.

The comparison occurs inside the inner loop:

IF Product[i].title = Product[j].title

AND Product[i].seller = Product[j].seller

We count one product-pair comparison per iteration, treating the title-and-seller check as a single basic operation.

Step 2: Derive the exact number of comparisons

The outer loop runs from $i = 0$to $n - 2$.

For a fixed i, the inner loop runs from $j = i + 1$to $n - 1$.

Therefore, the number of inner-loop iterations is:

$$(n - 1) - (i + 1) + 1 = n - i - 1$$

The total number of comparisons is:

$$C(n) = \sum_{i=0}^{n-2} (n - i - 1)$$

Expanding the summation:

$$C(n) = (n - 1) + (n - 2) + (n - 3) + \cdots + 2 + 1$$

Using the arithmetic-series formula:

$$1 + 2 + 3 + \cdots + k = \frac{k(k + 1)}{2}$$

where $k = n - 1$,

$$C(n) = \frac{(n - 1)n}{2}$$

Thus, the exact number of comparisons is:

$$\boxed{C(n) = \frac{n(n - 1)}{2}}$$

Step 3: Determine time complexity

Expanding the expression:

$$C(n) = \frac{n^2 - n}{2}$$
$$C(n) = \frac{1}{2}n^2 - \frac{1}{2}n$$

The dominant term is n^2.

Therefore:

$$\boxed{T(n) = \Theta(n^2)}$$

The algorithm takes quadratic time.

Step 4: Determine space complexity

The algorithm uses loop variables and processes each pair directly. It does not require an additional array proportional to n.

Assuming duplicate marking does not require additional storage beyond the existing product records:

$$\boxed{S(n) = \Theta(1)}$$

This is the auxiliary space complexity.

Interpretation

Doubling the catalogue size approximately quadruples the number of comparisons.

If $n = 1{,}000{,}000$:

$$C(n) = \frac{1{,}000{,}000(999{,}999)}{2}$$

$$\boxed{C(n) = 499{,}999{,}500{,}000}$$

This is approximately 500 billion comparisons.

M2: API Rate Limiter

The algorithm is:

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j])
   END FOR
   i ← i × 2
END WHILE
```

Step 1: Identify the basic operation

The basic operation is:

Audit(Request[j])

The inner loop executes $i$times during each outer-loop iteration.

Step 2: Determine the outer-loop values

The variable $i$doubles after each iteration:

$$1,2,4,8,16,\ldots,2^k$$

The loop condition is:

$$2^k < n$$

Taking logarithms:

$$k < \log_2 n$$

The largest valid exponent is:

$$k = \lceil \log_2 n \rceil - 1$$

Therefore, the outer loop executes:

$$\boxed{\lceil \log_2 n \rceil}$$

times for integer $n \geq 1$.

Step 3: Derive the exact number of audit operations

The inner-loop iteration counts are:

$$1 + 2 + 4 + 8 + \cdots + 2^{\lceil \log_2 n \rceil - 1}$$

Let:

$$m = \lceil \log_2 n \rceil$$

Then:

$$A(n) = \sum_{k=0}^{m-1} 2^k$$

Using the geometric-series formula:

$$1 + 2 + 4 + \cdots + 2^{m-1} = 2^m - 1$$

Therefore:

$$\boxed{A(n) = 2^{\lceil \log_2 n \rceil} - 1}$$

This formula also works when $n = 1$, producing zero audit operations.

Step 4: Determine the tight asymptotic bound

From the ceiling definition:

$$\log_2 n \leq \lceil \log_2 n \rceil < \log_2 n + 1$$

Taking powers of two:

$$n \leq 2^{\lceil \log_2 n \rceil} < 2n$$

Subtracting one:

$$n - 1 \leq A(n) < 2n - 1$$

Therefore:

$$\boxed{A(n) = \Theta(n)}$$

The team member's $\Theta(n \log n)$claim is incorrect.

The inner loop does not execute $n$times during every outer iteration. Its size doubles, producing a geometric series.

Step 5: Verify for $n = 1,024$

Since:

$$1{,}024 = 2^{10}$$

The outer-loop values are:

$$1,2,4,8,16,32,64,128,256,512$$

There are 10 iterations.

The total number of audit operations is:

$$A(1,024) = 1 + 2 + 4 + \cdots + 512$$
$$A(1,024) = 2^{10} - 1$$
$$\boxed{A(1,024) = 1{,}023}$$

Conclusion: M2 has linear time complexity, $\Theta(n)$, and does not require redesign solely because of asymptotic growth.

(c) Solve a Recurrence by Substitution

Product Search — Version S2

The recurrence is:

$$T(n) = T(n/2) + 1$$

with:

$$T(1) = 1$$

We solve it using backward substitution.

Step 1: Expand once

Starting with:

$$T(n) = T(n/2) + 1$$

Substitute the recurrence into $T(n/2)$:

$$T(n/2) = T(n/4) + 1$$

Therefore:

$$T(n) = T(n/4) + 1 + 1$$
$$\boxed{T(n) = T(n/4) + 2}$$

Step 2: Expand again

$$T(n/4) = T(n/8) + 1$$

Substituting:

$$T(n) = T(n/8) + 3$$

Step 3: Continue expanding

The pattern is:

$$T(n) = T(n/2) + 1$$
$$T(n) = T(n/4) + 2$$
$$T(n) = T(n/8) + 3$$

After $k$expansions:

$$\boxed{T(n) = T(n/2^k) + k}$$

Step 4: Find the stopping condition

The recurrence reaches its base case when:

$$\frac{n}{2^k} = 1$$

Therefore:

$$n = 2^k$$

Taking logarithms:

$$\log_2 n = k$$

Substitute $k = \log_2 n$:

$$T(n) = T(1) + \log_2 n$$

Since $T(1) = 1$:

$$\boxed{T(n) = 1 + \log_2 n}$$

This exact formula assumes $n$is a power of two and that each comparison is counted as one operation under the given recurrence.

For arbitrary positive integer n, the number of comparisons is:

$$\boxed{T(n) = \lfloor \log_2 n \rfloor + 1}$$

assuming the recurrence is interpreted with integer subproblem sizes and the standard binary-search halving process.

Step 5: Determine time complexity

The dominant term is $\log_2 n$.

Therefore:

$$\boxed{T(n) = \Theta(\log n)}$$

Step 6: Determine space complexity

Iterative implementation:

The search maintains a few variables, such as low, high, and mid.

$$\boxed{S_{\text{iterative}}(n) = \Theta(1)}$$

Recursive implementation:

Each recursive call uses a stack frame. There are logarithmically many calls.

$$\boxed{S_{\text{recursive}}(n) = \Theta(\log n)}$$

Interpretation

Binary search scales efficiently because it eliminates approximately half of the remaining products at each step.

For one million products:

$$\log_2(1,000,000) \approx 19.93$$

Thus, a successful or unsuccessful search requires approximately 20 comparisons, depending on the exact stopping condition and target position.

(d) Apply the Master Theorem

M4: Recommendation Engine

The Master Theorem applies to recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

where:

- $a$is the number of recursive subproblems.
- $b$is the factor by which the problem size is reduced.
- $f(n)$is the non-recursive work.

We compare the current design with the proposed redesign.

M4(a): Current Design

The recurrence is:

$$T(n) = 4T(n/2) + n$$

Step 1: Identify the parameters

$$a = 4,\ b = 2,\ f(n) = n$$

Step 2: Calculate $n^{\log_b a}$

$$n^{\log_b a} = n^{\log_2 4}$$

Since:

$$\log_2 4 = 2$$

we obtain:

$$\boxed{n^{\log_b a} = n^2}$$

Step 3: Compare $f(n)$with $n^{\log_b a}$

We have:

$$f(n) = n$$

and:

$$n^{\log_b a} = n^2$$

Therefore:

$$f(n) = O(n^{2-1})$$

Thus:

$$f(n) = O(n^1)$$

The non-recursive work grows polynomially slower than n^2.

Step 4: Apply the Master Theorem

Master Theorem Case 1 applies.

If:

$$f(n) = O\left(n^{\log_b a - \epsilon}\right)$$

for some $\epsilon > 0$, then:

$$T(n) = \Theta\left(n^{\log_b a}\right)$$

Here, $\epsilon = 1$.

Therefore:

$$\boxed{T(n) = \Theta(n^2)}$$

Step 5: Interpret the result

The current recommendation engine has quadratic time complexity.

If the input doubles, the running time grows approximately by:

$$\frac{T(2n)}{T(n)} \approx \frac{(2n)^2}{n^2} = 4$$

Doubling the input approximately quadruples the running time.

M4(b): Proposed Redesign

The recurrence becomes:

$$T(n) = 2T(n/2) + n$$

Step 1: Identify the parameters

$$a = 2,\ b = 2,\ f(n) = n$$

Step 2: Calculate $n^{\log_b a}$

$$n^{\log_2 2} = n^1$$

Therefore:

$$\boxed{n^{\log_b a} = n}$$

Step 3: Compare the functions

$$f(n) = n$$

and:

$$n^{\log_b a} = n$$

Hence:

$$f(n) = \Theta\left(n^{\log_b a}\right)$$

Step 4: Apply the Master Theorem

Master Theorem Case 2 applies.

If:

$$f(n) = \Theta\left(n^{\log_b a}\right)$$

then:

$$T(n) = \Theta\left(n^{\log_b a} \log n\right)$$

Therefore:

$$\boxed{T(n) = \Theta(n \log n)}$$

Step 5: Interpret the result

When the input doubles, the running time grows by approximately:

$$\frac{T(2n)}{T(n)} \approx \frac{2n\log_2(2n)}{n\log_2 n}$$

Since:

$$\log_2(2n) = \log_2 n + 1$$

we obtain:

$$\frac{T(2n)}{T(n)} \approx 2\,\frac{\log_2 n + 1}{\log_2 n}$$

For large n, this ratio approaches 2.

Doubling the input approximately doubles the running time, with a small additional logarithmic factor.

Comparison of M4 Designs

Feature	**Current design**	**Proposed redesign**
Recurrence	$4T(n/2) + n$	$2T(n/2) + n$
a	4	2
b	2	2
$f(n)$	n	n
$n^{\log_b a}$	n^2	n
Master Theorem case	Case 1	Case 2

Feature	Current design	Proposed redesign
Time complexity	$\Theta(n^2)$	$\Theta(n \log n)$
Effect of doubling input	Approximately 4 ×time	Approximately 2 ×time

Recommendation: The proposed redesign is significantly more scalable because it eliminates unnecessary replication and reduces the asymptotic complexity from quadratic to $n \log n$.

M5: Sales Analytics Engine

The recurrence is:

$$T(n) = 2T(n/2) + n \log n$$

Step 1: Identify the parameters

$$a = 2,\ b = 2,\ f(n) = n \log n$$

Step 2: Calculate $n^{\log_b a}$

$$n^{\log_2 2} = n$$

Therefore:

$$\boxed{n^{\log_b a} = n}$$

Step 3: Compare $f(n)$with $n^{\log_b a}$

We have:

$$f(n) = n \log n$$

and:

$$n^{\log_b a} = n$$

Therefore:

$$f(n) = \Theta(n \log n)$$

This is larger than the equal-growth condition in standard Case 2.

The three basic Master Theorem cases, in their usual forms, do not directly provide the result for this logarithmic factor. We can use the extended Master Theorem or a recursion tree.

Step 4: Apply the extended Master Theorem

The extended Case 2 states that if:

$$f(n) = \Theta\left(n^{\log_b a} \log^k n\right)$$

then:

$$T(n) = \Theta\left(n^{\log_b a}\log^{k+1} n\right)$$

Here:

$$k = 1$$

Therefore:

$$T(n) = \Theta(n\log^{1+1} n)$$

Hence:

$$\boxed{T(n) = \Theta(n\log^2 n)}$$

Alternative derivation: Recursion tree

At the root, the non-recursive work is:

$$n\log_2 n$$

At level 1, there are two subproblems of size $n/2$.

Total work at level 1:

$$2\left(\frac{n}{2}\log_2\frac{n}{2}\right)$$

Since:

$$\log_2\frac{n}{2} = \log_2 n - 1$$

the total is:

$$n(\log_2 n - 1)$$

At level 2, there are four subproblems of size $n/4$.

Total work:

$$4\left(\frac{n}{4}\log_2\frac{n}{4}\right)$$
$$= n(\log_2 n - 2)$$

At level i, the total work is:

$$W_i = n(\log_2 n - i)$$

The tree has approximately $\log_2 n$levels.

Summing the work across the levels:

$$T(n) = \sum_{i=0}^{\log_2 n - 1} n\,(\log_2 n - i)$$

Let:

$$L = \log_2 n$$

Then:

$$T(n) = n \sum_{i=0}^{L-1} (L - i)$$

Rearranging:

$$T(n) = n(1 + 2 + \cdots + L)$$

Using the arithmetic-series formula:

$$T(n) = n\frac{L(L+1)}{2}$$

Substituting $L = \log_2 n$:

$$T(n) = \frac{n}{2}\log_2 n(\log_2 n + 1)$$

Therefore, the dominant term is:

$$\boxed{T(n) = \Theta(n\log^2 n)}$$

Interpretation: The sales analytics engine is less scalable than an $n \log n$algorithm. Its extra logarithmic factor can significantly increase processing time for a large catalogue.

(e) Analyse Best, Average and Worst Cases

M3–S1: Sequential Search

Sequential search checks products from the beginning until the target is found.

Let $n$be the number of products.

Best case

Condition: The target is the first product.

Number of comparisons:

$$\boxed{C_{\text{best}} = 1}$$

Time complexity:

$$\boxed{\Theta(1)}$$

Average case

Assume the target exists and is equally likely to appear at any of the $n$positions.

If the target is at position i, it requires $i$comparisons.

Therefore:

$$C_{\text{avg}} = \frac{1 + 2 + \cdots + n}{n}$$

Using the arithmetic-series formula:

$$C_{\text{avg}} = \frac{n(n+1)}{2n}$$

Hence:

$$\boxed{C_{\text{avg}} = \frac{n+1}{2}}$$

Time complexity:

$$\boxed{\Theta(n)}$$

Worst case

Condition: The target is the last product or is absent from the catalogue.

Number of comparisons:

$$\boxed{C_{\text{worst}} = n}$$

Time complexity:

$$\boxed{\Theta(n)}$$

Derive the Average Comparisons When the Target Exists with Probability p

We are told that the probability of the target existing is p.

Assume:

- If the target exists, it is equally likely to appear at any position.
- If the target does not exist, the algorithm scans all $n$products.

When the target exists, the average number of comparisons is:

$$C_{\text{found}} = \frac{n+1}{2}$$

When the target does not exist:

$$C_{\text{absent}} = n$$

Using the weighted-average formula:

$$E[C] = pC_{\text{found}} + (1-p)C_{\text{absent}}$$

Substituting:

$$E[C] = p\left(\frac{n+1}{2}\right) + (1-p)n$$

Expanding:

$$E[C] = \frac{p(n+1)}{2} + n - pn$$

Taking the common denominator:

$$E[C] = \frac{p(n+1) + 2n - 2pn}{2}$$

Simplifying:

$$\boxed{E[C] = n - \frac{p(n-1)}{2}}$$

Evaluate for $n = 1,000,000$ and $p = 0.5$

Substitute the given values:

$$E[C] = 1{,}000{,}000 - \frac{0.5(1{,}000{,}000 - 1)}{2}$$
$$E[C] = 1{,}000{,}000 - \frac{0.5(999{,}999)}{2}$$
$$E[C] = 1{,}000{,}000 - 249{,}999.75$$

Therefore:

$$\boxed{E[C] = 750{,}000.25}$$

The expected number of comparisons is approximately **750,000 per search**.

This is far more than the approximately 20 comparisons required by binary search.

M3–S2: Binary Search

Binary search requires the product index to be sorted.

Best case

Condition: The target matches the middle element on the first comparison.

$$\boxed{C_{\text{best}} = 1}$$

Average case

The exact average depends on the target's position and the distribution of searched keys. Under a uniform distribution over a sorted array, the number of comparisons is logarithmic.

$$\boxed{C_{\text{avg}} = \Theta(\log n)}$$

Worst case

Condition: The target is found at the deepest search level or is absent and the algorithm reaches its stopping condition.

For the standard binary-search halving process:

$$\boxed{C_{\text{worst}} = \lfloor \log_2 n \rfloor + 1}$$

Thus:

$$\boxed{C_{\text{worst}} = \Theta(\log n)}$$

For $n = 1{,}000{,}000$:

$$\lfloor \log_2(1,000,000) \rfloor + 1 = 19 + 1$$

Therefore:

$$\boxed{C_{\text{worst}} = 20}$$

Comparison of the Search Versions

Case	**Sequential search S1**	**Binary search S2**
Best	1 comparison	1 comparison
Average	$(n+1)/2$, if target exists uniformly	$\Theta(\log n)$
Worst	$n$comparisons	$\lfloor \log_2 n \rfloor + 1$
Time complexity	Best: $\Theta(1)$; average/worst: $\Theta(n)$	Best: $\Theta(1)$; average/worst: $\Theta(\log n)$
Requirement	No sorting needed	Sorted index required

Recommendation: Use binary search for frequent product lookups when a sorted index can be maintained efficiently.

(f) Prove an Asymptotic Bound

We must prove that:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

is:

$$\boxed{f(n) = \Theta(n^2)}$$

using the formal definitions.

Step 1: Recall the definition of Θ

A function $f(n)$is $\Theta\big(g(n)\big)$if there exist positive constants $c_1, c_2$and an integer $n_0$such that:

$$0 \le c_1 g(n) \le f(n) \le c_2 g(n)$$

for every $n \ge n_0$.

Here:

$$g(n) = n^2$$

We must establish both the lower bound and the upper bound.

Step 2: Prove the lower bound

Given:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

For $n \ge 1$:

$$10n\log_2 n \ge 0$$

and:

$$50 > 0$$

Therefore:

$$f(n) \ge 3n^2$$

Choose:

$$\boxed{c_1 = 3}$$

Then:

$$\boxed{3n^2 \le f(n)}$$

for all $n \ge 1$.

Step 3: Prove the upper bound

For $n \ge 1$, the standard inequality:

$$\log_2 n \le n$$

holds.

Multiplying both sides by n:

$$n\log_2 n \leq n^2$$

Therefore:

$$10n\log_2 n \leq 10n^2$$

Also, for $n \geq 1$:

$$50 \leq 50n^2$$

Adding the terms:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$
$$f(n) \leq 3n^2 + 10n^2 + 50n^2$$
$$f(n) \leq 63n^2$$

Choose:

$$\boxed{c_2 = 63}$$

Thus:

$$\boxed{f(n) \leq 63n^2}$$

for all $n \geq 1$.

Step 4: State the constants

We have established:

$$3n^2 \leq f(n) \leq 63n^2$$

for all $n \geq 1$.

Therefore, valid constants are:

$$\boxed{c_1 = 3,\ c_2 = 63,\ n_0 = 1}$$

By the formal definition:

$$\boxed{f(n) = \Theta(n^2)}$$

Prove That $f(n)$Is Not $O(n)$

By definition, $f(n)$is $O(n)$only if there exist constants $c > 0$and $n_0$such that:

$$f(n) \leq cn$$

for all $n \geq n_0$.

But:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

Since all terms are non-negative for $n \geq 1$:

$$f(n) \geq 3n^2$$

Suppose $f(n) = O(n)$. Then some constant $c > 0$must satisfy:

$$3n^2 \leq f(n) \leq cn$$

Therefore:

$$3n^2 \leq cn$$

Dividing by $n > 0$:

$$3n \leq c$$

Hence:

$$n \leq \frac{c}{3}$$

This cannot hold for every sufficiently large n, because $n$can increase without bound while $c$is a fixed constant.

Therefore, our assumption is false.

$$\boxed{f(n) \notin O(n)}$$

Conclusion: The report generator's cost is quadratic because the n^2term dominates the lower-order terms as $n$grows.

(g) Scalability Decision

We estimate the operation counts and running times for $n = 1{,}000{,}000$.

Given:

$$n = 1{,}000{,}000$$

Server speed:

$$R = 10^8 \text{ operations/second}$$

The running-time estimate is:

$$\boxed{\text{Time in seconds} = \frac{\text{Number of operations}}{10^8}}$$

The maximum number of operations allowed within 10 minutes is:

$$10^8 \times 600 = 60{,}000{,}000{,}000$$

Thus, a module requiring more than 60 billion operations exceeds the assumed nightly processing budget.

Important assumption: The estimates below treat the specified recurrence or comparison count as the basic-operation count. Actual execution times may differ because memory access, I/O, database operations, and the cost of individual comparisons are not necessarily equal.

1. M1: Duplicate-Listing Detector

Exact comparisons:

$$C(n) = \frac{n(n-1)}{2}$$

Substituting:

$$C(1{,}000{,}000) = \frac{1{,}000{,}000(999{,}999)}{2}$$

$$\boxed{C(n) = 499{,}999{,}500{,}000}$$

Running time:

$$t = \frac{499{,}999{,}500{,}000}{100{,}000{,}000}$$

$$\boxed{t = 4{,}999.995 \text{ seconds}}$$

Converting to minutes:

$$\frac{4{,}999.995}{60} \approx 83.33 \text{ minutes}$$

Result: Approximately 83 minutes and 20 seconds.

Decision: M1 exceeds the 10-minute limit and must be redesigned.

Recommended redesign

Use a hash-based approach:

1. Construct a composite key from the normalized product title and seller ID.
2. Store the key in a hash set or hash map.
3. If the key already exists, mark the listing as a duplicate.
4. Otherwise, insert the key.

Expected time complexity:

$$\boxed{\Theta(n)}$$

This is expected linear time under appropriate hashing assumptions, although the worst case can be worse if collisions are excessive.

2. M2: API Rate Limiter

The exact audit count is:

$$A(n) = 2^{\lceil \log_2 n \rceil} - 1$$

For $n = 1{,}000{,}000$:

$$\lceil \log_2(1,000,000) \rceil = 20$$

Therefore:

$$A(n) = 2^{20} - 1$$
$$\boxed{A(n) = 1{,}048{,}575}$$

Running time:

$$t = \frac{1{,}048{,}575}{10^8}$$
$$\boxed{t = 0.01048575 \text{ seconds}}$$

Approximately:

$$\boxed{10.49 \text{ milliseconds}}$$

Result: M2 easily meets the nightly processing limit.

Decision: No redesign is required based on asymptotic complexity. However, the implementation should still be checked for request-window correctness, memory use, and the cost of the audit operation.

3. M3–S2: Binary Search

The worst-case comparison count is:

$$C(n) = \lfloor \log_2 n \rfloor + 1$$

For $n = 1{,}000{,}000$:

$$C(n) = 19 + 1$$
$$\boxed{C(n) = 20}$$

Running time under the one-operation-per-comparison model:

$$t = \frac{20}{10^8}$$

$$\boxed{t = 0.0000002 \text{ seconds}}$$

This equals:

$$\boxed{0.2 \text{ microseconds}}$$

Result: Binary search easily meets the limit.

Decision: No redesign is required for search complexity. The product index must remain sorted, and index maintenance costs should be considered separately.

4. M4: Current Recommendation Engine

The recurrence is:

$$T(n) = 4T(n/2) + n$$

Its asymptotic complexity is:

$$T(n) = \Theta(n^2)$$

The exact operation count depends on the base case. To provide a concrete estimate, assume:

$$T(1) = 1$$

and that the recurrence holds exactly for powers of two.

For this recurrence:

$$T(n) = 4T(n/2) + n$$

Expanding:

$$T(n) = 4\left[4T(n/4) + \frac{n}{2}\right] + n$$
$$T(n) = 16T(n/4) + 2n + n$$
$$T(n) = 16T(n/4) + 3n$$

After $k$levels:

$$T(n) = 4^k T(n/2^k) + kn$$

Set:

$$n/2^k = 1$$

Thus:

$$k = \log_2 n$$

Since $4^k = n^2$and $T(1) = 1$:

$$T(n) = n^2 + n\log_2 n$$

Therefore, under this specific base-case assumption:

$$\boxed{T(n) = n^2 + n\log_2 n}$$

For $n = 1{,}000{,}000$:

$$T(n) = 10^{12} + 1{,}000{,}000\log_2(1{,}000{,}000)$$

Using:

$$\log_2(1{,}000{,}000) \approx 19.9316$$

we get:

$$T(n) \approx 10^{12} + 19{,}931{,}569$$
$$\boxed{T(n) \approx 1{,}000{,}000{,}019{,}931{,}569}$$

Running time:

$$t \approx \frac{1{,}000{,}000{,}019{,}931{,}569}{10^8}$$
$$\boxed{t \approx 10{,}000{,}000.20 \text{ seconds}}$$

In days:

$$\frac{10{,}000{,}000.20}{86{,}400} \approx 115.74$$

Result: Approximately 116 days under this simplified operation-count model.

Decision: The current M4 design fails the nightly window by a very large margin and must be redesigned.

5. M4: Proposed Recommendation Engine

The recurrence is:

$$T(n) = 2T(n/2) + n$$

Assume again:

$$T(1) = 1$$

For powers of two, expansion gives:

$$T(n) = 2T(n/2) + n$$
$$= 4T(n/4) + 2n$$
$$= 8T(n/8) + 3n$$

After $k$levels:

$$T(n) = 2^k T(n/2^k) + kn$$

Set:

$$k = \log_2 n$$

Since $2^k = n$and $T(1) = 1$:

$$T(n) = n + n\log_2 n$$

Therefore:

$$\boxed{T(n) = n(1 + \log_2 n)}$$

For $n = 1{,}000{,}000$:

$$T(n) = 1{,}000{,}000(1 + 19.9316)$$
$$\boxed{T(n) \approx 20{,}931{,}569}$$

Running time:

$$t = \frac{20{,}931{,}569}{10^8}$$
$$\boxed{t \approx 0.2093 \text{ seconds}}$$

Result: Approximately 0.21 seconds.

Decision: The redesigned M4 meets the nightly limit under the stated model.

The improvement is substantial because the asymptotic complexity changes from quadratic to $n \log n$.

6. M5: Sales Analytics Engine

The recurrence is:

$$T(n) = 2T(n/2) + n\log_2 n$$

The complexity is:

$$\boxed{T(n) = \Theta(n\log^2 n)}$$

To estimate the count, assume $n$is a power of two and use a recursion tree with zero cost at size 1.

At each level i, the total non-recursive work is:

$$W_i = n(\log_2 n - i)$$

Let:

$$L = \log_2 n$$

Summing across the Lnon-leaf levels:

$$T(n) = n\sum_{i=0}^{L-1}(L - i)$$
$$T(n) = n(1 + 2 + \cdots + L)$$

Using the arithmetic-series formula:

$$T(n) = n\frac{L(L+1)}{2}$$

Therefore:

$$\boxed{T(n) = \frac{n}{2}\log_2 n(\log_2 n + 1)}$$

For $n = 1{,}000{,}000$:

$$L \approx 19.9316$$

Then:

$$T(n) \approx \frac{1{,}000{,}000}{2}(19.9316)(20.9316)$$
$$\boxed{T(n) \approx 208{,}580{,}000}$$

Running time:

$$t \approx \frac{208{,}580{,}000}{10^8}$$
$$\boxed{t \approx 2.09 \text{ seconds}}$$

Result: Approximately 2.09 seconds under the simplified recurrence model.

Decision: M5 meets the nightly window under the stated assumptions, although its extra logarithmic factor makes it less scalable than an $n \log n$alternative.

Summary of Scalability Estimates

Module	**Approximate operations for $n = 1{,}000{,}000$**	**Estimated time**	**Meets 10-minute limit?**
M1: Duplicate detector	4.999995×10^{11}	83.33 minutes	**No**
M2: API rate limiter	1,048,575	10.49 milliseconds	Yes
M3–S2: Binary search	20 comparisons	0.2 microseconds	Yes

Module	Approximate operations for $n = 1{,}000{,}000$	Estimated time	Meets 10-minute limit?
M4: Current design	$1.00000002 \times 10^{15}$	Approximately 116 days	**No**
M4: Proposed design	20,931,569	0.21 seconds	Yes
M5: Sales analytics	Approximately 208,580,000	2.09 seconds	Yes

Important qualification: M4's and M5's exact counts depend on the recurrence's base case and on how the merge, sorting, and other operations are counted. The numerical estimates use the explicit assumptions described above. Actual production timings may differ.

Final Scalability Recommendations

The audit identifies two immediate priorities.

1. Redesign M1: Duplicate-Listing Detector

M1 requires approximately 500 billion comparisons at one million products, corresponding to about 83 minutes under the given processing model.

Replace pairwise comparison with a hash-based duplicate-detection strategy.

Expected complexity changes from:

$$\Theta(n^2) \longrightarrow \Theta(n)$$

This is the most direct improvement for the duplicate detector.

2. Redesign M4: Recommendation Engine

The current design has quadratic complexity:

$$\Theta(n^2)$$

The proposed design has:

$$\Theta(n \log n)$$

Removing unnecessary replication reduces the estimated operation count dramatically and brings the module within the nightly processing budget under the given model.

3. Retain M2, S2, and M5 with monitoring

- **M2:** Its geometric-series iteration count is linear, not $n \log n$. No redesign is required solely on complexity grounds.

- **S2:** Binary search performs approximately 20 comparisons for one million products. Maintain a sorted index and account for index-update costs.
- **M5:** Its $\Theta(n\log^2 n)$complexity meets the current 10-minute limit under the simplified model. Monitor real processing time as the sales dataset grows and consider an $n \log n$sorting approach if the implementation can be improved without changing required functionality.

Overall Conclusion

The ShopSphere scalability audit demonstrates why asymptotic analysis is important when planning for rapid data growth. M1's pairwise comparison algorithm and M4's replicated recursive design have quadratic growth and should be redesigned before the catalogue reaches one million products. M2 has linear growth, S2 has logarithmic search complexity, and M5 has $\Theta(n\log^2 n)$complexity. Under the stated operation-rate assumption and recurrence models, the proposed M4 redesign, M2, S2, and M5 fit within the 10-minute nightly window.

The primary engineering recommendation is to replace M1's pairwise comparisons with expected-linear-time hashing and adopt the proposed M4 redesign to eliminate unnecessary recursive replication. These changes provide the greatest improvements in scalability and reduce the risk of nightly processing delays.